

Презентация по росту дендритов

Этап 4, Результаты проекта

Вакутайпа Милдред, Разанацуа Сара Естэлл, Нджову Нелиа

2026-05-16

Содержание I

- 1 Информация
- 2 Вводная часть
- 3 Теоретическое описание задачи
- 4 Описание модели
- 5 Алгоритм
- 6 Построение модели

Раздел 1

Информация

- Вакутайпа Милдред



- Вакутайпа Милдред
- НКНбд-01-23



- Вакутайпа Милдред
- НКНбд-01-23
- студентка кафедры математического моделирования и искусственного интеллекта



- Вакутайпа Милдред
- НКНбд-01-23
- студентка кафедры математического моделирования и искусственного интеллекта
- Российский университет дружбы народов им. П. Лумумбы



- Вакутайпа Милдред
- НКНбд-01-23
- студентка кафедры математического моделирования и искусственного интеллекта
- Российский университет дружбы народов им. П. Лумумбы
- 1032239009@rudn.ru



- Вакутайпа Милдред
- НКНбд-01-23
- студентка кафедры математического моделирования и искусственного интеллекта
- Российский университет дружбы народов им. П. Лумумбы
- 1032239009@rudn.ru
- <https://wakutaipa.github.io/>



Раздел 2

Вводная часть

- Процессы кристаллизации и роста дендритов играют ключевую роль в различных областях науки и техники

- Процессы кристаллизации и роста дендритов играют ключевую роль в различных областях науки и техники
- Дендритная микроструктура, возникающая при неравновесном затвердевании, непосредственно влияет на распределение примесей, образование пор и трещин, а также на последующую термическую обработку материала.

- Дендриты

Объект и предмет исследования

- Дендриты
- Кристаллические дендриты

- 1 Построить модель роста дендритов

Цели и задачи

- 1 Построить модель роста дендритов
- 2 Создать алгоритм модели роста дендритов

Цели и задачи

- 1 Построить модель роста дендритов
- 2 Создать алгоритм модели роста дендритов
- 3 Моделировать процесс с помощью языка программирования

Раздел 3

Теоретическое описание задачи

$$\rho c_p \frac{\partial T}{\partial t} = \kappa \nabla^2 T \equiv \kappa \left(\frac{\partial^2 T}{\partial x^2} + \frac{\partial^2 T}{\partial y^2} \right)$$

- Основной причиной образования дендритных ветвей

Неустойчивость Муллинса – Секерки

- Основной причиной образования дендритных ветвей
- Плоская граница затвердевания является неустойчивой

- Соотношение Гиббса – Томсона:

$$T_b = T_m \left(1 - \frac{\gamma T_m}{\rho L^2 R} \right)$$

$$d_0 = \frac{\gamma T_m c_p}{\rho L^2}$$

- Соотношение Гиббса – Томсона:

$$T_b = T_m \left(1 - \frac{\gamma T_m}{\rho L^2 R} \right)$$

- Капиллярный радиус:

$$d_0 = \frac{\gamma T_m c_p}{\rho L^2}$$

$$\Delta T_b = -T_m \beta V$$

Безразмерная форма

Безразмерная температура определяется как:

$$\tilde{T} = \frac{c_p(T - T_\infty)}{L}$$

Параметр безразмерного переохлаждения:

$$S = \frac{c_p(T_m - T_\infty)}{L}$$

В безразмерной форме:

$$\frac{\partial \tilde{T}}{\partial t} = \chi \nabla^2 \tilde{T}$$

Эффект Гиббса – Томсона в безразмерной форме:

$$T_b = \tilde{T}_m - \lambda \cdot (\text{член кривизны})$$

Раздел 4

Описание модели

- Модель реализуется на двумерной квадратной сетке размером $N \times N$ узлов

Общая структура модели

- Модель реализуется на двумерной квадратной сетке размером $N \times N$ узлов
- Расстояние между соседними узлами h

Общая структура модели

- Модель реализуется на двумерной квадратной сетке размером $N \times N$ узлов
- Расстояние между соседними узлами h
- В центре области задается твердая затравка

- Модель реализуется на двумерной квадратной сетке размером $N \times N$ узлов
- Расстояние между соседними узлами h
- В центре области задается твердая затравка
- Каждый узел сетки может находиться в одном из двух состояний: жидком или твердом

Численная схема для уравнения теплопроводности

- Лапласиан $\nabla^2 T$ в узле (i, j) аппроксимируется через значения температуры в соседних узлах:

$$\nabla^2 T \approx \frac{\langle T_{(i,j)} \rangle - T_{i,j}}{(4 + 4w)(1 + 2w)h^2}$$

$$\langle T_{(i,j)} \rangle = \frac{(T_{i+1,j} + T_{i-1,j} + T_{i,j+1} + T_{i,j-1}) + w(T_{i+1,j+1} + T_{i+1,j-1} + T_{i-1,j+1} + T_{i-1,j-1})}{4 + 4w}$$

Численная схема для уравнения теплопроводности

- Лапласиан $\nabla^2 T$ в узле (i, j) аппроксимируется через значения температуры в соседних узлах:

$$\nabla^2 T \approx \frac{\langle T_{(i,j)} \rangle - T_{i,j}}{(4 + 4w)(1 + 2w)h^2}$$

- Среднее значение температуры соседей $\langle T_{(i,j)} \rangle$ вычисляется с учетом как ближайших, так и диагональных соседей:

$$\langle T_{(i,j)} \rangle = \frac{(T_{i+1,j} + T_{i-1,j} + T_{i,j+1} + T_{i,j-1}) + w(T_{i+1,j+1} + T_{i+1,j-1} + T_{i-1,j+1} + T_{i-1,j-1})}{4 + 4w}$$

Численная схема для уравнения теплопроводности

- Лапласиан $\nabla^2 T$ в узле (i, j) аппроксимируется через значения температуры в соседних узлах:

$$\nabla^2 T \approx \frac{\langle T_{(i,j)} \rangle - T_{i,j}}{(4 + 4w)(1 + 2w)h^2}$$

- Среднее значение температуры соседей $\langle T_{(i,j)} \rangle$ вычисляется с учетом как ближайших, так и диагональных соседей:

$$\langle T_{(i,j)} \rangle = \frac{(T_{i+1,j} + T_{i-1,j} + T_{i,j+1} + T_{i,j-1}) + w(T_{i+1,j+1} + T_{i+1,j-1} + T_{i-1,j+1} + T_{i-1,j-1})}{4 + 4w}$$

- $0 \leq w \leq 1$ - вклад диагональных соседей

Численная схема для уравнения теплопроводности

- Явная схема обновления температуры:

$$T_{i,j}^{new} = T_{i,j} + \chi \Delta t \nabla^2 T$$

$$\frac{\chi \Delta t}{mh^2} < \frac{1}{4}$$

Численная схема для уравнения теплопроводности

- Явная схема обновления температуры:

$$T_{i,j}^{new} = T_{i,j} + \chi \Delta t \nabla^2 T$$

- Для обеспечения устойчивости схемы и корректного учета разномасштабности

$$T_{i,j}^{new} = T_{i,j} + \chi \frac{\Delta t}{m} \nabla^2 T$$

$$\frac{\chi \Delta t}{m h^2} < \frac{1}{4}$$

Численная схема для уравнения теплопроводности

- Явная схема обновления температуры:

$$T_{i,j}^{new} = T_{i,j} + \chi \Delta t \nabla^2 T$$

- Для обеспечения устойчивости схемы и корректного учета разномасштабности

$$T_{i,j}^{new} = T_{i,j} + \chi \frac{\Delta t}{m} \nabla^2 T$$

- Условие устойчивости явной схемы:

$$\frac{\chi \Delta t}{m h^2} < \frac{1}{4}$$

- Для учета эффекта Гиббса – Томсона необходимо вычислять локальную кривизну границы

$$s_{i,j} = \sum_{\text{ближайшие}} n_{i,j} + w_n \sum_{\text{диагональные}} n_{i,j} - \left(\frac{5}{2} + \frac{5}{2} w_n \right)$$

- Для учета эффекта Гиббса – Томсона необходимо вычислять локальную кривизну границы
- Выражение для дискретизированной кривизны имеет вид:

$$s_{i,j} = \sum_{\text{ближайшие}} n_{i,j} + w_n \sum_{\text{диагональные}} n_{i,j} - \left(\frac{5}{2} + \frac{5}{2} w_n \right)$$

- Условие затвердевания учитывает все физические механизмы:

$$T \leq \tilde{T}_m(1 + \eta_{i,j}\delta) + \lambda s_{i,j}$$

Раздел 5

Алгоритм

Алгоритм моделирования роста дендритов состоит из следующих основных этапов:

- 1 Запуск программы

Алгоритм моделирования роста дендритов состоит из следующих основных этапов:

- 1 Запуск программы
- 2 Шаг 1: Настройка параметров модели

Алгоритм моделирования роста дендритов состоит из следующих основных этапов:

- 1 Запуск программы
- 2 Шаг 1: Настройка параметров модели
- 3 Шаг 2: Инициализация массивов и проверка условия остановки

Алгоритм моделирования роста дендритов состоит из следующих основных этапов:

- 1 Запуск программы
- 2 Шаг 1: Настройка параметров модели
- 3 Шаг 2: Инициализация массивов и проверка условия остановки
- 4 Шаг 3: Расчет температурного поля

Алгоритм моделирования роста дендритов состоит из следующих основных этапов:

- 1 Запуск программы
- 2 Шаг 1: Настройка параметров модели
- 3 Шаг 2: Инициализация массивов и проверка условия остановки
- 4 Шаг 3: Расчет температурного поля
- 5 Шаг 4: Моделирование роста, сохранение результатов, возврат к проверке условия остановки, завершение моделирования

Алгоритм моделирования роста дендритов состоит из следующих основных этапов:

- 1 Запуск программы
- 2 Шаг 1: Настройка параметров модели
- 3 Шаг 2: Инициализация массивов и проверка условия останова
- 4 Шаг 3: Расчет температурного поля
- 5 Шаг 4: Моделирование роста, сохранение результатов, возврат к проверке условия останова, завершение моделирования
- 6 Шаг 5: Организация основного цикла вычислений

Алгоритм моделирования роста дендритов состоит из следующих основных этапов:

- 1 Запуск программы
- 2 Шаг 1: Настройка параметров модели
- 3 Шаг 2: Инициализация массивов и проверка условия останова
- 4 Шаг 3: Расчет температурного поля
- 5 Шаг 4: Моделирование роста, сохранение результатов, возврат к проверке условия останова, завершение моделирования
- 6 Шаг 5: Организация основного цикла вычислений
- 7 Шаг 6: Исследование зависимости характеристик роста от параметров

Алгоритм моделирования роста дендритов состоит из следующих основных этапов:

- 1 Запуск программы
- 2 Шаг 1: Настройка параметров модели
- 3 Шаг 2: Инициализация массивов и проверка условия останова
- 4 Шаг 3: Расчет температурного поля
- 5 Шаг 4: Моделирование роста, сохранение результатов, возврат к проверке условия останова, завершение моделирования
- 6 Шаг 5: Организация основного цикла вычислений
- 7 Шаг 6: Исследование зависимости характеристик роста от параметров
- 8 Шаг 7: Визуализация результатов

Раздел 6

Построение модели

Задание базовых параметров моделирования

```
N = 150  
T_initial = -1  
steps = 200  
dt = 1  
h = 1  
kappa = 0.1  
w = 0.5  
T_m = 0  
lambda = 0.01  
delta = 0.02
```

Инициализация сетки

```
T = zeros(N, N)
n = zeros(Int, N, N)
T[N/2+1, N/2+1] = T_initial
n[N/2+1, N/2+1] = 1
```

- Метод полиномиальной аппроксимации

```
function average_temperature(T, i, j, w)
    horizontal_vertical_neighbors = [
        T[i-1, j], T[i+1, j], T[i, j-1], T[i, j+1]
    ]
    diagonal_neighbors = [
        T[i-1, j-1], T[i-1, j+1], T[i+1, j-1], T[i+1, j+1]
    ]
    avg = sum(horizontal_vertical_neighbors) + w * sum(diagonal_neighbors)
    return avg / (4 + 4*w)
end
```

Рисунок 1: Функция average_temperature

- Метод полиномиальной аппроксимации
- Среднее значение температуры

```
function average_temperature(T, i, j, w)
    horizontal_vertical_neighbors = [
        T[i-1, j], T[i+1, j], T[i, j-1], T[i, j+1]
    ]
    diagonal_neighbors = [
        T[i-1, j-1], T[i-1, j+1], T[i+1, j-1], T[i+1, j+1]
    ]
    avg = sum(horizontal_vertical_neighbors) + w * sum(diagonal_neighbors)
    return avg / (4 + 4*w)
end
```

Рисунок 1: Функция average_temperature

- Метод полиномиальной аппроксимации
- Среднее значение температуры
- Кривизна границы

```
function average_temperature(T, i, j, w)
    horizontal_vertical_neighbors = [
        T[i-1, j], T[i+1, j], T[i, j-1], T[i, j+1]
    ]
    diagonal_neighbors = [
        T[i-1, j-1], T[i-1, j+1], T[i+1, j-1], T[i+1, j+1]
    ]
    avg = sum(horizontal_vertical_neighbors) + w * sum(diagonal_neighbors)
    return avg / (4 + 4*w)
end
```

Рисунок 1: Функция average_temperature

- Метод полиномиальной аппроксимации
- Среднее значение температуры
- Кривизна границы
- Количества затвердевших частиц

```
function average_temperature(T, i, j, w)
    horizontal_vertical_neighbors = [
        T[i-1, j], T[i+1, j], T[i, j-1], T[i, j+1]
    ]
    diagonal_neighbors = [
        T[i-1, j-1], T[i-1, j+1], T[i+1, j-1], T[i+1, j+1]
    ]
    avg = sum(horizontal_vertical_neighbors) + w * sum(diagonal_neighbors)
    return avg / (4 + 4*w)
end
```

Рисунок 1: Функция average_temperature

- Метод полиномиальной аппроксимации
- Среднее значение температуры
- Кривизна границы
- Количества затвердевших частиц
- Среднеквадратичный радиус

```
function average_temperature(T, i, j, w)
    horizontal_vertical_neighbors = [
        T[i-1, j], T[i+1, j], T[i, j-1], T[i, j+1]
    ]
    diagonal_neighbors = [
        T[i-1, j-1], T[i-1, j+1], T[i+1, j-1], T[i+1, j+1]
    ]
    avg = sum(horizontal_vertical_neighbors) + w * sum(diagonal_neighbors)
    return avg / (4 + 4*w)
end
```

Рисунок 1: Функция average_temperature

- Метод полиномиальной аппроксимации

```
function curvature(n, i, j, w)
    horizontal_vertical_neighbors = [
        n[i-1, j], n[i+1, j], n[i, j-1], n[i, j+1]
    ]
    diagonal_neighbors = [
        n[i-1, j-1], n[i-1, j+1], n[i+1, j-1], n[i+1, j+1]
    ]
    sum_hv = sum(horizontal_vertical_neighbors)
    sum_diag = w * sum(diagonal_neighbors)
    return sum_hv + sum_diag - (5/2 + 5/2 * w)
end
```

Рисунок 2: Функция curvate

- Метод полиномиальной аппроксимации
- Среднее значение температуры

```
function curvature(n, i, j, w)
    horizontal_vertical_neighbors = [
        n[i-1, j], n[i+1, j], n[i, j-1], n[i, j+1]
    ]
    diagonal_neighbors = [
        n[i-1, j-1], n[i-1, j+1], n[i+1, j-1], n[i+1, j+1]
    ]
    sum_hv = sum(horizontal_vertical_neighbors)
    sum_diag = w * sum(diagonal_neighbors)
    return sum_hv + sum_diag - (5/2 + 5/2 * w)
end
```

Рисунок 2: Функция curvate

- Метод полиномиальной аппроксимации
- Среднее значение температуры
- Кривизна границы

```
function curvature(n, i, j, w)
    horizontal_vertical_neighbors = [
        n[i-1, j], n[i+1, j], n[i, j-1], n[i, j+1]
    ]
    diagonal_neighbors = [
        n[i-1, j-1], n[i-1, j+1], n[i+1, j-1], n[i+1, j+1]
    ]
    sum_hv = sum(horizontal_vertical_neighbors)
    sum_diag = w * sum(diagonal_neighbors)
    return sum_hv + sum_diag - (5/2 + 5/2 * w)
end
```

Рисунок 2: Функция curvate

- Метод полиномиальной аппроксимации
- Среднее значение температуры
- Кривизна границы
- Количества затвердевших частиц

```
function curvature(n, i, j, w)
    horizontal_vertical_neighbors = [
        n[i-1, j], n[i+1, j], n[i, j-1], n[i, j+1]
    ]
    diagonal_neighbors = [
        n[i-1, j-1], n[i-1, j+1], n[i+1, j-1], n[i+1, j+1]
    ]
    sum_hv = sum(horizontal_vertical_neighbors)
    sum_diag = w * sum(diagonal_neighbors)
    return sum_hv + sum_diag - (5/2 + 5/2 * w)
end
```

Рисунок 2: Функция curvate

- Метод полиномиальной аппроксимации
- Среднее значение температуры
- Кривизна границы
- Количества затвердевших частиц
- Среднеквадратичный радиус

```
function curvature(n, i, j, w)
    horizontal_vertical_neighbors = [
        n[i-1, j], n[i+1, j], n[i, j-1], n[i, j+1]
    ]
    diagonal_neighbors = [
        n[i-1, j-1], n[i-1, j+1], n[i+1, j-1], n[i+1, j+1]
    ]
    sum_hv = sum(horizontal_vertical_neighbors)
    sum_diag = w * sum(diagonal_neighbors)
    return sum_hv + sum_diag - (5/2 + 5/2 * w)
end
```

Рисунок 2: Функция curvate

- Метод полиномиальной аппроксимации

```
function count_solid_particles(n)
    return sum(n)
end

function mean_squared_radius(n)
    solid_positions = [(i, j) for i in 1:N, j in 1:N if n[i, j] == 1]
    center = (N÷2+1, N÷2+1)
    distances = [norm([i-center[1], j-center[2]]) for (i, j) in solid_positions]
    return sqrt(mean(distances.^2))
end
```

Рисунок 3: Функции count_solid_particles и mean_squared_radius

- Метод полиномиальной аппроксимации
- Среднее значение температуры

```
function count_solid_particles(n)
    return sum(n)
end

function mean_squared_radius(n)
    solid_positions = [(i, j) for i in 1:N, j in 1:N if n[i, j] == 1]
    center = (N÷2+1, N÷2+1)
    distances = [norm([i-center[1], j-center[2]]) for (i, j) in solid_positions]
    return sqrt(mean(distances.^2))
end
```

Рисунок 3: Функции count_solid_particles и mean_squared_radius

- Метод полиномиальной аппроксимации
- Среднее значение температуры
- Кривизна границы

```
function count_solid_particles(n)
    return sum(n)
end

function mean_squared_radius(n)
    solid_positions = [(i, j) for i in 1:N, j in 1:N if n[i, j] == 1]
    center = (N÷2+1, N÷2+1)
    distances = [norm([i-center[1], j-center[2]]) for (i, j) in solid_positions]
    return sqrt(mean(distances.^2))
end
```

Рисунок 3: Функции count_solid_particles и mean_squared_radius

- Метод полиномиальной аппроксимации
- Среднее значение температуры
- Кривизна границы
- Количества затвердевших частиц

```
function count_solid_particles(n)
    return sum(n)
end

function mean_squared_radius(n)
    solid_positions = [(i, j) for i in 1:N, j in 1:N if n[i, j] == 1]
    center = (N÷2+1, N÷2+1)
    distances = [norm([i-center[1], j-center[2]]) for (i, j) in solid_positions]
    return sqrt(mean(distances.^2))
end
```

Рисунок 3: Функции count_solid_particles и mean_squared_radius

- Метод полиномиальной аппроксимации
- Среднее значение температуры
- Кривизна границы
- Количества затвердевших частиц
- Среднеквадратичный радиус

```
function count_solid_particles(n)
    return sum(n)
end

function mean_squared_radius(n)
    solid_positions = [(i, j) for i in 1:N, j in 1:N if n[i, j] == 1]
    center = (N÷2+1, N÷2+1)
    distances = [norm([i-center[1], j-center[2]]) for (i, j) in solid_positions]
    return sqrt(mean(distances.^2))
end
```

Рисунок 3: Функции count_solid_particles и mean_squared_radius

Модель теплопроводности

Функция `simulate_heat_conduction` на основе уравнения обновления температуры:

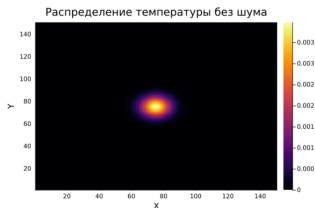


Рисунок 4: Распределение температуры без шума

```
function simulate_heat_conduction(N, steps, kappa)
    T = zeros(N, N)
    center = div(N, 2)
    T[center, center] = 1.0 # начальная температура в центре

    for step in 1:steps
        T_temp = copy(T)
        for i in 2:N-1
            for j in 2:N-1
                T_temp[i, j] = T[i, j] + kappa * (T[i+1, j] + T[i-1, j] + T[i, j+1] + T[i, j-1] - 4 * T[i, j])
            end
        end
        T .= T_temp
    end

    heatmap(T, title="Распределение температуры без шума", xlabel="X", ylabel="Y")
end
```

Рисунок 5: Функция `simulate_heat_conduction`

Добавление процесса затвердевания

Реализована функция `simulate_solidification`, которая выполняет следующие шаги:

1 Обновление температур

```
function simulate_solidification(T, n, steps, w, kappa, dt, h, delta, T_m, lambda)
    solid_counts = []
    mean_radaii = []
    fractal_dims = []
    # Основной цикл моделирования
    for step in 1:steps
        T_temp = copy(T) # Создаем временную копию для текущего шага
        n_temp = copy(n) # Создаем временную копию для состояний

        # Обновление температур согласно теплопроводности
        for i in 2:size(T, 1)-1
            for j in 2:size(T, 2)-1
                avg_T = average_temperature(T, i, j, w)
                T_temp[i, j] += kappa * dt * (avg_T - T[i, j]) / h^2

                # Добавление случайного теплового шума
                eta_ij = rand(-1.0:0.01:1.0) # Случайное число [-1, 1]
                T_temp[i, j] += eta_ij * delta
            end
        end
    end
```

Рисунок 6: Фрагмент функции `simulate_solidification`

Добавление процесса затвердевания

Реализована функция `simulate_solidification`, которая выполняет следующие шаги:

- 1 Обновление температур
- 2 Проверка условия затвердевания

```
function simulate_solidification(T, n, steps, w, kappa, dt, h, delta, T_m, lambda)
    solid_counts = []
    mean_radaii = []
    fractal_dims = []
    # Основной цикл моделирования
    for step in 1:steps
        T_temp = copy(T) # Создаем временную копию для текущего шага
        n_temp = copy(n) # Создаем временную копию для состояний

        # Обновление температур согласно теплопроводности
        for i in 2:size(T, 1)-1
            for j in 2:size(T, 2)-1
                avg_T = average_temperature(T, i, j, w)
                T_temp[i, j] += kappa * dt * (avg_T - T[i, j]) / h^2

                # Добавление случайного теплового шума
                eta_ij = rand(-1.0:0.01:1.0) # Случайное число [-1, 1]
                T_temp[i, j] += eta_ij * delta
            end
        end
    end
end
```

Рисунок 6: Фрагмент функции `simulate_solidification`

Добавление процесса затвердевания

Реализована функция `simulate_solidification`, которая выполняет следующие шаги:

- 1 Обновление температур
- 2 Проверка условия затвердевания
- 3 Обновление состояний

```
function simulate_solidification(T, n, steps, w, kappa, dt, h, delta, T_m, lambda)
    solid_counts = []
    mean_radii = []
    fractal_dims = []
    # Основной цикл моделирования
    for step in 1:steps
        T_temp = copy(T) # Создаем временную копию для текущего шага
        n_temp = copy(n) # Создаем временную копию для состояний

        # Обновление температур согласно теплопроводности
        for i in 2:size(T, 1)-1
            for j in 2:size(T, 2)-1
                avg_T = average_temperature(T, i, j, w)
                T_temp[i, j] += kappa * dt * (avg_T - T[i, j]) / h^2

                # Добавление случайного теплового шума
                eta_ij = rand(-1.0:0.01:1.0) # Случайное число [-1, 1]
                T_temp[i, j] += eta_ij * delta
            end
        end
    end
```

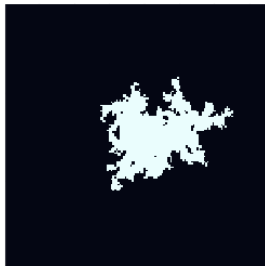
Рисунок 6: Фрагмент функции `simulate_solidification`

Результаты моделирования. Исследование влияния капиллярного радиуса

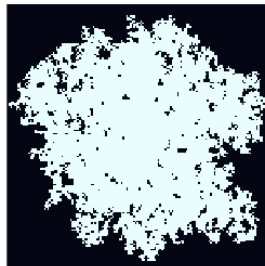
$T=-1, \lambda=0.05$



$T=-1, \lambda=0.01$



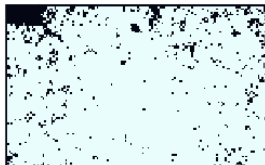
$T=-1, \lambda=0.005$



$T=-1, \lambda=0.001$



$T=-1, \lambda=0.0001$



Динамика роста агрегата

Зависимость количества затвердевших частиц от времени

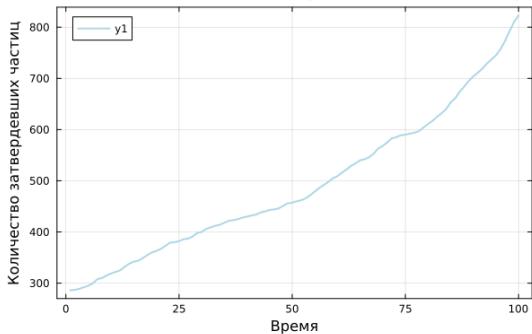


Рисунок 8: Зависимость числа затвердевших частиц от времени

Зависимость среднеквадратического радиуса от времени

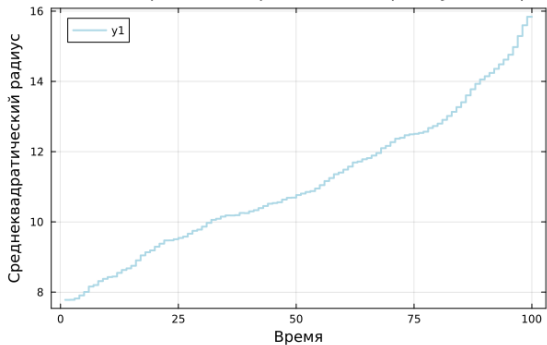


Рисунок 9: Зависимость среднеквадратического радиуса от времени

Фрактальная размерность

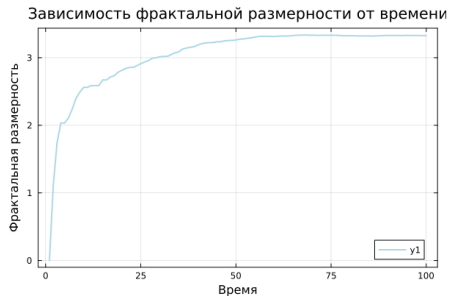


Рисунок 10: Зависимость фрактальной размерности от времени

Фрактальную размерность D можно определить через логарифмическую регрессию:

$$D = \frac{\log N(r)}{\log r} \quad ()$$

где:

- $N(r)$ - количество частиц внутри радиуса r

Реализована функция `fractal_dimension`

Фрактальная размерность

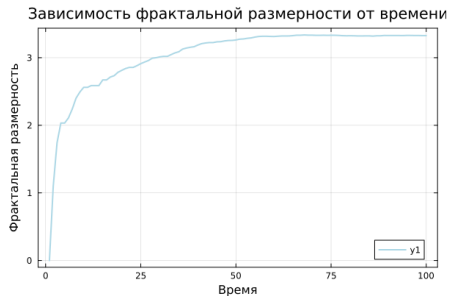


Рисунок 10: Зависимость фрактальной размерности от времени

Фрактальную размерность D можно определить через логарифмическую регрессию:

$$D = \frac{\log N(r)}{\log r} \quad ()$$

где:

- $N(r)$ - количество частиц внутри радиуса r
- D - искомая фрактальная размерность

Реализована функция `fractal_dimension`

Влияние теплового шума

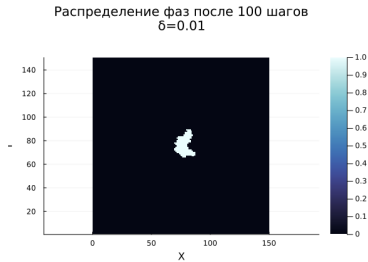


Рисунок 11: Значение теплового шума (δ) 0.01



Рисунок 12: Значение теплового шума (δ) 0.05

Во время выполнения группового проекта выполнены следующие задания:

- описана модель роста дендритов

Во время выполнения группового проекта выполнены следующие задания:

- описана модель роста дендритов
- описан алгоритм из 7 шагов для моделирования роста дендритов

Во время выполнения группового проекта выполнены следующие задания:

- описана модель роста дендритов
- описан алгоритм из 7 шагов для моделирования роста дендритов
- Смоделирован процесс теплопроводности.

Во время выполнения группового проекта выполнены следующие задания:

- описана модель роста дендритов
- описан алгоритм из 7 шагов для моделирования роста дендритов
- Смоделирован процесс теплопроводности.
- Исследовано влияние начального переохлаждения и капиллярного радиуса на форму дендритов.

Во время выполнения группового проекта выполнены следующие задания:

- описана модель роста дендритов
- описан алгоритм из 7 шагов для моделирования роста дендритов
- Смоделирован процесс теплопроводности.
- Исследовано влияние начального переохлаждения и капиллярного радиуса на форму дендритов.
- Проанализирована динамика роста агрегата и его фрактальная размерность.

Во время выполнения группового проекта выполнены следующие задания:

- описана модель роста дендритов
- описан алгоритм из 7 шагов для моделирования роста дендритов
- Смоделирован процесс теплопроводности.
- Исследовано влияние начального переохлаждения и капиллярного радиуса на форму дендритов.
- Проанализирована динамика роста агрегата и его фрактальная размерность.
- Изучено влияние теплового шума на морфологию агрегатов.